L4 GPU Precision Formats on Colab

https://gemini.google.com/app/e1010b6c11ba8ff2

User prompt: What is the best precision format for the L4 GPU on Google Colab?I have access to it on the Colab plan.

Response: The optimal precision format for the NVIDIA L4 GPU on Google Colab depends on whether your workload is **training/fine-tuning** or **inference**, but the standout format enabled by its architecture is **FP8**.

The NVIDIA L4 is built on the **Ada Lovelace architecture** (Compute Capability 8.9) with 4th-generation Tensor Cores and 24 GB of GDDR6 VRAM.

Workload Recommendations

Workload	Recommended Format	Why It's Best	Alternative / Fallback
LLM / Vision Inference	**FP8 (`fp8_e4m3` / `fp8_e5m2`)**	2× the throughput of FP16/BF16, cuts memory bandwidth bottleneck, and maintains output accuracy better than INT8.	**INT4 / AWQ / GPTQ** (for fitting 14B–32B models into 24 GB VRAM).
Fine-Tuning / Training (LoRA, QLoRA)	**BF16 (`bfloat16`)**	Native Ada Lovelace support, identical dynamic range to FP32, eliminates underflow/overflow gradients without dynamic loss scaling.	**Mixed FP16** (if specific older model layers or custom CUDA kernels lack BF16 support).
Quantized Training / QLoRA Base Weights	**4-bit NF4 (`bitsandbytes`)** with **BF16 compute**	Keeps weight footprint minimal (~5.5 GB for 7B–8B models) while doing matrix operations at full 16-bit precision.	**8-bit (`int8`)** base weights.

Key Precisions on the L4

1. FP8 (The True Advantage of L4 over T4)

Unlike the older Turing-based T4 (which only supported FP16/INT8/INT4), the L4 supports native hardware FP8 (E4M3 and E5M2).

- **When to use:** Serving models using vLLM, TensorRT-LLM, or PyTorch 2.x Transformer Engine / `torch.ao.quantization`.
- **Benefit:** FP8 roughly doubles compute TFLOPS compared to FP16 while fitting 13B–14B models completely uncompressed into the 24 GB VRAM without noticeable degradation in perplexity.

2. BF16 (`bfloat16`)

On older GPUs like the T4, attempting `bfloat16` triggers software emulation and crawls. On the L4:

- **Native Tensor Core support:** Full hardware-accelerated BF16 execution.
- **Stability:** BF16 shares the 8-bit exponent of FP32, preventing `NaN` or loss spikes common in FP16 fine-tuning runs.

3. INT4 / W4A16 (AWQ / GPTQ)

- **When to use:** When serving larger models (e.g., 14B, 27B, or 32B parameters) that physically cannot fit in 24 GB VRAM in FP16 or FP8.
- **Benefit:** 4-bit weights packed into memory with on-the-fly dequantization to FP16/BF16 Tensor Cores during the forward pass.

Implementation Snippets for Colab

Hugging Face Transformers / PEFT (LoRA Fine-tuning):

```
import torch
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "model id",
```

```
    torch_dtype=torch.bfloat16,  # Native to L4; avoid torch.float16 if BF16 is supported
    device_map="auto"
)
```

vLLM Inference with FP8 Quantization:

```
from vllm import LLM, SamplingParams

# Load pre-quantized FP8 models directly on L4
llm = LLM(
    model="neuralmagic/Meta-Llama-3.1-8B-Instruct-FP8",
    kv_cache_dtype="fp8",
    gpu_memory_utilization=0.9
)
```

bitsandbytes 4-Bit QLoRA:

```
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,  # Leverage L4's fast BF16 compute cores
    bnb_4bit_use_double_quant=True,
)
```

Sources

1. NVIDIA L4 Tensor Core GPU: Smart AI Inference & Video Workloads
2. NVIDIA L4 GPU specs: detailed specifications explained - CloudPe

---

User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?

Response: To maximize token generation speed on the NVIDIA L4, the best precision format depends on whether you are optimizing for **single-stream speed (latency / batch size = 1)** or **concurrent throughput (serving multiple requests / large batch size)**.

The Verdict

- **Single Request / Interactive Chat (Batch Size = 1):** Use **4-bit Weight-Only Quantization (W4A16 via Marlin or AWQ)**.
- **Concurrent / High-Volume Generation (Batch Size $\geq$ 8–16):** Use **FP8 (W8A8 with FP8 KV-Cache)**.

Why the Bottleneck Changes

Token generation occurs in two phases with different hardware bottlenecks: **prefill** (processing the prompt) and **decoding** (generating tokens one by one). Autoregressive decoding is where token generation speed is determined.

```
+-------------------------------------------------------------------------------------+
| Single Stream (Batch = 1)   --> Bounded by MEMORY BANDWIDTH (300 GB/s GDDR6)         |
| Large Batch (Batch >= 16)   --> Bounded by TENSOR CORE COMPUTE (FLOPs / TOPS)        |
+-------------------------------------------------------------------------------------+
```

1. Single Stream Speed: Memory Bandwidth Bound (Choose W4A16 Marlin)

When generating one token for a single user, the GPU must stream **every single model weight from VRAM into the compute cores for every single token produced**.

The L4 GPU has a memory bandwidth of **300 GB/s**. The theoretical upper bound for tokens per second (T) at batch size 1 is:

$$T \approx \frac{\text{Memory Bandwidth (bytes/sec)}}{\text{Active Model Size in VRAM (bytes)}}$$

For an **8B parameter model**:

- **BF16 / FP16 (16 GB):** $300\ \text{GB/s} / 16\ \text{GB} \approx \mathbf{18}$ to $\mathbf{19}\ \text{tokens/s}$ max theoretical.
- **FP8 (8 GB):** $300\ \text{GB/s} / 8\ \text{GB} \approx \mathbf{37}$ to $\mathbf{38}\ \text{tokens/s}$ max theoretical.
- **INT4 / W4A16 (4.5 GB):** $300\ \text{GB/s} / 4.5\ \text{GB} \approx \mathbf{66}$ to $\mathbf{68}\ \text{tokens/s}$ max theoretical.

Using **4-bit weights** cuts the data transferred over the memory bus by up to 75% compared to 16-bit. Pair this with a high-performance dequantization kernel like **Marlin** (supported natively in vLLM), which dequantizes 4-bit weights to 16-bit registers on-the-fly without compute stall.

2. Batched Throughput: Compute Bound (Choose FP8 W8A8)

When processing multiple prompts or a high concurrency batch, the GPU reuses weights across requests, escaping the memory bandwidth bottleneck and hitting the limit of the Tensor Cores.

The L4's Ada Lovelace architecture features 4th-generation Tensor Cores with native FP8 engines:

- **BF16 / FP16 Tensor Core:** 121 TFLOPS (dense)
- **FP8 Tensor Core:** 242 TFLOPS (dense) — **2× raw compute throughput**

Under heavy batching:

- **4-bit weight-only schemes** fall behind because the actual matrix multiplication still runs at 16-bit precision after dequantization.
- **FP8** executes matrix multiplications directly on the 242 TFLOPS FP8 Tensor Cores without dequantization overhead.
- Enabling an **FP8 KV-Cache** cuts KV memory usage in half, allowing twice the batch size inside the L4's 24 GB VRAM before running out of memory.

Performance Comparison (8B Model on NVIDIA L4)

Precision Format	Target Bottleneck	Single Stream (Tokens/sec)	Max Batched Throughput	Context/VRAM Headroom
BF16	Baseline	~15–18 tok/s	Baseline	Low (~16 GB model alone)
FP8 (W8A8)	Compute & KV Bandwidth	~30–35 tok/s	**Highest (up to 2.5× BF16)**	High (~8 GB model)
AWQ / GPTQ (W4A16)	Memory Bandwidth	~40–48 tok/s	Medium	Very High (~4.5 GB model)
Marlin (W4A16)	Memory Bandwidth	**Highest (~55–62 tok/s)**	High	Very High (~4.5 GB model)

Recommended vLLM Setup on Colab

To get the absolute highest token generation rate on an L4 instance, run via **vLLM** using either optimized format:

For Maximum Single-Stream Speed (W4A16 with Marlin kernel):

```
from vllm import LLM, SamplingParams

# Load an AWQ/GPTQ model; vLLM automatically routes through the fast Marlin kernel on L4
llm = LLM(
    model="casperhansen/llama-3-8b-instruct-awq",
    quantization="marlin",       # Optimized 4-bit kernel for Ada Lovelace
    kv_cache_dtype="auto",
    gpu_memory_utilization=0.9
)
```

For Maximum Batched Throughput (Native FP8 + FP8 KV-Cache):

```
from vllm import LLM, SamplingParams

# Pure FP8 pipeline leveraging Ada Lovelace 242 TFLOPS Tensor Cores
llm = LLM(
    model="neuralmagic/Meta-Llama-3.1-8B-Instruct-FP8",
    kv_cache_dtype="fp8",        # FP8 KV-cache doubles batch capacity
    gpu_memory_utilization=0.9
)
```
